

Sistemas de Big Data

*Conforme a contenidos del «Curso de Especialización
en Inteligencia Artificial y Big Data»*



Universidad de Castilla-La Mancha

Escuela Superior de Informática
Ciudad Real

Capítulo 9

Análisis de datos en Big Data

José Ángel Martín Baos

Hasta el momento, hemos visto como podemos utilizar diferentes técnicas de Big Data para la obtención, el almacenamiento y el procesamiento de la información contenida en grandes conjuntos de datos. En los siguientes cuatro capítulos estudiaremos diferentes técnicas que nos van a permitir tratar estos datos, analizarlos y visualizarlos para extraer de ellos información de interés para el negocio.

Existen numerosas técnicas que permiten analizar un problema desde diversas perspectivas, dependiendo de las características del problema y de los resultados que se quieran obtener. Estas técnicas varían desde métricas muy sencillas hasta técnicas complejas que requieren de un gran poder computacional para ser llevadas a cabo. En este capítulo nos centraremos en las principales técnicas de análisis de datos empleadas de manera tradicional, mientras que en el próximo capítulo se describirán técnicas de inteligencia artificial en el análisis de datos.

Las técnicas que estudiaremos a lo largo de estos capítulos se ejemplificarán mediante el uso del lenguaje de programación Python junto con algunos de los paquetes y bibliotecas más relevantes actualmente y que pueden utilizarse en este lenguaje para el análisis de datos y su posterior visualización.

También se empleará para el análisis de datos la herramienta Apache Spark a través de su interfaz para Python denominada PySpark. Spark es uno de los frameworks más ampliamente utilizados en el área del Big Data. Este framework de computación en cluster nos permitirá realizar análisis avanzados de grandes volúmenes de datos utilizando un gran número de librerías preconfiguradas, fáciles y rápidas de usar.

9.1. Introducción a Apache Spark

Tal y como se describió en el Capítulo 3, Apache Spark¹ es un framework de programación de código abierto usado para el procesamiento batch, streaming y tiempo real de datos distribuidos. Este framework fue desarrollado inicialmente por la Universidad de California y donado más tarde a la Apache Software Foundation bajo licencia open-source Apache License 2.0. Su interfaz de programación permite trabajar con paralelismo de datos y tolerancia a fallos mediante el uso de clusters de computadores. Spark proporciona APIs de programación para Java, Scala, Python y R. También soporta el procesamiento de datos estructurados basados en el lenguaje SQL. Hay un gran número de bibliotecas integradas en Spark, incluida la biblioteca MLlib para el aprendizaje automático, lo cual constituye una de sus características principales.

Spark constituye una de las mejores alternativas disponibles para el procesamiento de datos distribuido. En el momento de la redacción de este libro, Apache Spark cuenta con más de 30000 estrellas (*stars*) en la plataforma Github, siendo uno de los proyectos open source más activos de Big Data. Actualmente Spark ha alcanzado un alto grado de madurez y es una de las herramientas con mayor nivel de adopción corporativo. Es, posiblemente, el motor de procesamiento más popular en Big Data. Por este motivo, es una de las opciones que primero debe tenerse en cuenta a la hora de desarrollar una aplicación de Big Data.

Spark forma parte del ecosistema Hadoop que hemos estudiado en los capítulos previos. Una de las características más importantes de Spark es que soporta una alta integración con el sistema de ficheros distribuidos del framework Hadoop (HDFS) lo que permite que pueda integrarse dentro de clusters YARN con sistema de archivos HDFS. Por ese motivo es bastante frecuente encontrar Spark sobre HDFS en entornos donde ya existen aplicaciones basadas en MapReduce como PIG o HIVE. En este contexto, Spark permite complementar la funcionalidad existente o incluso reemplazar los programas previamente implementados en MapReduce, mejorando la eficiencia de estos.

No obstante, Spark también permite ser instalado y ejecutado en máquinas individuales o clusters sin YARN ni HDFS, constituyendo por si mismo un grupo de herramientas autosuficiente. En este caso Spark trabaja directamente y sin intermediarios sobre las fuentes de datos existentes.

Spark está basado en un motor computacional que se encarga de la programación, la distribución y la supervisión de las aplicaciones que se ejecutan. Cada tarea se distribuye entre varios nodos de trabajo del cluster de computación denominados *workers*. Al nodo que gestiona y coordina todas las tareas se la denomina *master* o *manager*. Todas las tareas realizadas por los nodos workers se agregan al final para producir una única salida.

¹ Enlace: <http://spark.apache.org/>

9.2. Creación de un cluster Apache Spark en Docker

Para ejecutar una aplicación en Spark, lo primero que se realiza es crear un objeto *SparkContext* dentro del programa principal. Este objeto se conectará a un proceso denominado *cluster manager* y coordinará todos los procesos ejecutados en el cluster de Spark. El cluster manager se encargará de proporcionar los recursos necesarios a las aplicaciones. Hay varios tipos de cluster managers a los que Spark-Context puede conectarse, ya sea a un propio cluster de Spark o bien a un cluster YARN. Una vez conectado, Spark genera procesos *executors* en aquellos nodos del cluster que el cluster manager le haya proporcionado, y a continuación les envía a estos procesos el código de la aplicación. Estos procesos executors se encargaran de ejecutar los cálculos requeridos por el programa, así como almacenar los datos de la aplicación. Finalmente, SparkContext envía a los executors las tareas que deben ejecutar (*tasks*). Esta arquitectura está ilustrada en la figura 9.1.

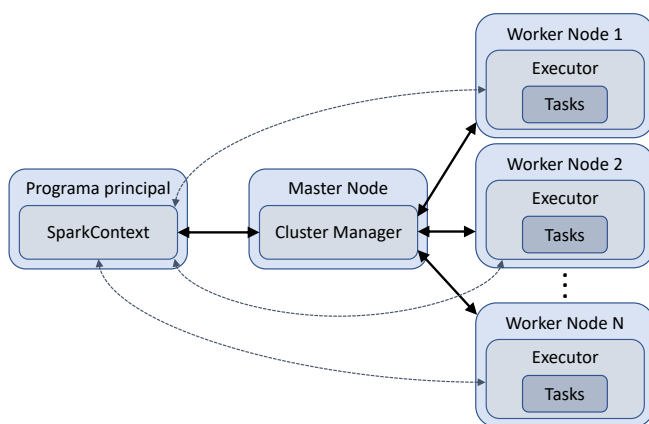


Figura 9.1: Arquitectura de un cluster de Spark

En este curso estudiaremos el uso de Spark dentro del entorno de programación Python. La biblioteca de Python para usar Spark se denomina **PySpark**. Abordaremos el uso de las características principales de PySpark para el análisis de datos, así como de la biblioteca MLlib para la ejecución iterativa de métodos de aprendizaje automático.

9.2. Creación de un cluster Apache Spark en Docker

En esta sección aprenderemos como construir un pequeño cluster de Apache Spark para ilustrar su funcionamiento. Para construir este cluster necesitamos al menos un nodo master y una serie de nodos workers. También se hará uso de un nodo ejecutando JupyterLab, una aplicación web que permite crear cuadernos interactivos de Jupyter en Python y ejecutarlos en la propia máquina en la que se ejecuta el servidor web.

Se hará uso de Docker para crear este cluster de ejemplo, ya que el uso de máquinas virtuales sería más costoso computacionalmente y ocuparía una gran cantidad de disco duro en la máquina del lector. Jupyter ofrece una imagen de Docker con Apache Spark instalado y lista para usar con una interfaz de JupyterLab, sin embargo esta imagen ejecuta un único contenedor y no nos permitirá mostrar la ejecución en un entrono distribuido. Por este motivo, a lo largo de esta sección se explicarán los pasos necesarios para crear una serie de imágenes de Docker que nos permitirán definir nuestro cluster. Este cluster simulará el uso de tres máquinas diferentes (nodos) mediante el uso de tres contenedores en Docker. También se creará una imagen con una interfaz de JupyterLab para la programación de las aplicaciones.

9.2.1. Fundamentos de Docker

Docker ofrece a los desarrolladores una herramienta económica y muy confiable para poder crear, distribuir y desplegar aplicaciones distribuidas y complejas en una gran variedad de entornos, proporcionando una capa de abstracción gracias a la cual no es necesario tener en cuenta el sistema operativo que ejecuta por debajo de este. En los últimos años, Docker se ha convertido en la tecnología de contenedores más utilizada, ya que implementa una API que permite ejecutar contenedores livianos de manera aislada.



Contenedor. Un contenedor es una unidad de software que incluye todo lo necesario para que una aplicación se ejecute, incluyendo el código fuente, las bibliotecas, y el resto de dependencias necesarias para su correcta ejecución en cualquier entorno.

En este libro únicamente se darán unas nociones básicas de Docker necesarias para construir el cluster de Spark. El lector puede profundizar sobre el uso de Docker en su web oficial² o en libros especializados como [Tur14] o [Pou20].

En primer lugar será necesaria la instalación de Docker en la máquina del usuario. Si esta cuenta con Linux (que es el sistema operativo recomendado para ejecutar todos los programas explicados en este capítulo) se debe instalar **Docker Engine**³. En este libro se asumirá que el usuario tiene una máquina con el sistema operativo Ubuntu. Por lo tanto, se instalará Docker Engine mediante los comandos:

```
$ sudo apt update
$ sudo apt install docker-ce
```

No obstante, para sistemas basados en Windows o MacOS se debe instalar la alternativa **Docker Desktop**, la cual incluye Docker Engine y una interfaz gráfica, entre otras herramientas⁴.

² Enlace: <http://www.docker.com/>

³ Enlace: <http://docs.docker.com/engine/install/ubuntu/>

⁴ Enlace: <http://docs.docker.com/desktop/>

9.2. Creación de un cluster Apache Spark en Docker

Para ejecutar Docker sin privilegio de sudo, se debe añadir el usuario al grupo *docker* y después cerrar la sesión actual y volver a iniciar sesión.

```
$ sudo usermod -aG docker <tu-nombre-de-usuario>
```

También se debe instalar **Docker Compose**, que es una herramienta que se utilizará para definir y ejecutar aplicaciones con múltiples contenedores de manera sencilla. Para su instalación se pueden seguir los pasos explicados en el [🔗 Enlace: http://docs.docker.com/compose/install/](http://docs.docker.com/compose/install/).

Una vez configurado Docker en nuestra máquina, ya estamos listos para crear las imágenes y los contenedores que nos van a permitir definir nuestro cluster de Spark. Una **imagen** en Docker es un archivo estático, compuesto por múltiples capas, y que se utiliza como plantilla base para crear un contenedor. Los **contenedores** son instancias de las imágenes. De hecho, la principal diferencia entre los contenedores y las imágenes es que los contenedores son imágenes que tienen una capa de escritura en la cual se van añadiendo los cambios ocurridos una vez se ejecuta el contenedor.

9.2.2. Visión general del cluster

El cluster estará compuesto por cuatro nodos: un nodo Spark master, dos nodos Spark worker y un nodo con una interfaz de JupyterLab. El último nodo incluirá también la API de PySpark. Todos los nodos se conectarán mediante la red local y estarán montados sobre una simulación del sistema distribuidos de archivos de Hadoop HDFS, tal y como estarían montados en un cluster real. Para implementar cada nodo se utilizará un contenedor de Docker y para simular el sistema HDFS se montará un volumen de datos compartido entre todas los contenedores. Esta infraestructura está resumida de manera gráfica en la figura 9.2.

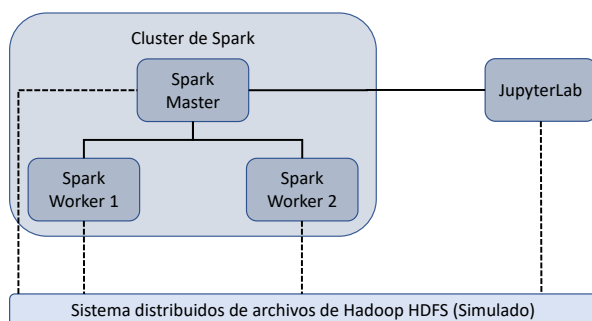


Figura 9.2: Visión general de nuestro cluster de Spark en Docker

El usuario se conectará a la interfaz de JupyterLab para ejecutar su programa en Python usando PySpark. Este programa se conectará al nodo master que procesará la entrada y distribuirá la carga computacional entre los nodos worker, que a su vez devolverán el resultado al nodo principal.

El sistema Spark soporta actualmente varios gestores de cluster. Utilizaremos el modo *standalone* de Spark que permite utilizar un sencillo gestor de clústeres incluido con Spark y que facilita la configuración del clúster.

9.2.3. Creación de las imágenes de Docker

A continuación se muestran las distintas imágenes que se han elaborado para montar este cluster. Se ha decidido tomar como imagen base para todas las imágenes una distribución de Linux muy ligera con Java 8 pre-instalado, ya que disponer de una distribución de Java 8 o superior es uno de los requisitos de Spark. Posteriormente se instalan las dependencias necesarias, como Python 3 (para ejecutar PySpark), Spark, Hadoop y JupyterLab (en aquellos nodos necesarios). Se han utilizado las versiones de Spark, Hadoop y JupyterLab estables y recomendadas a fecha de Octubre de 2021.

Spark master

El listado 9.1 muestra el fichero Docker que permite crear la imagen para el nodo Spark master. Este nodo instalará Python 3 y algunas bibliotecas de este lenguaje, así como Spark sobre Hadoop, montará una carpeta compartida que simulara el sistema HDFS y configura la imagen para ejecutar Spark como nodo master. Exponemos el puerto 8080 al exterior del contenedor, que es el que utilizaremos para conectarnos a una interfaz web de Spark para monitorizar el cluster.

Listado 9.1: Fichero spark-master.Dockerfile

```
1 # Selección de imagen base
2 # Especificamos como imagen base una imagen Debian ligera con Java 8 JRE
3 FROM openjdk:8-jre-slim
4
5
6 # Descarga e instalación de dependencias
7 # Definimos las variables del Dockerfile
8 ARG hdfs_simulado=/opt/workspace
9 ARG spark_version=3.1.2
10 ARG hadoop_version=3.2
11 ARG spark_master_web=8080 # Puerto para interfaz web del nodo master
12
13 # Definimos la variable de entorno conteniendo el directorio de HDFS
14 ENV HDFS_SIMULADO=${hdfs_simulado}
15
16 # Realizamos la instalación de la última versión estable de Python3
17 RUN mkdir -p ${hdfs_simulado} && \
18     apt-get update -y && \
19     apt-get install -y python3 && \
```


9.2. Creación de un cluster Apache Spark en Docker

```
20 ln -s /usr/bin/python3 /usr/bin/python && \
21 rm -rf /var/lib/apt/lists/*
22 RUN apt-get update -y && \
23     apt-get install -y python3-pip && \
24     pip3 install gdown numpy matplotlib scipy scikit-learn
25
26 # Realizamos la instalación de Apache Spark
27 RUN apt-get update -y && \
28     apt-get install -y curl && \
29     curl https://archive.apache.org/dist/spark/spark-${spark_version}/spark-${
        spark_version}-bin-hadoop${hadoop_version}.tgz -o spark.tgz && \
30     tar -xf spark.tgz && \
31     mv spark-${spark_version}-bin-hadoop${hadoop_version} /usr/bin/ && \
32     mkdir /usr/bin/spark-${spark_version}-bin-hadoop${hadoop_version}/logs && \
33     rm spark.tgz
34
35 # Definimos las variables de entorno de Spark
36 ENV SPARK_HOME /usr/bin/spark-${spark_version}-bin-hadoop${hadoop_version}
37 ENV SPARK_MASTER_HOST spark-master
38 ENV SPARK_MASTER_PORT 7077
39 ENV PYSPARK_PYTHON python3
40
41 # Exponemos el puerto para que los nodos worker conecten con el nodo master
42 EXPOSE ${SPARK_MASTER_PORT}
43 # Exponemos el puerto utilizado para acceder a la interfaz web del master
44 EXPOSE ${spark_master_web}
45
46
47 # Ejecución de comandos al arrancar el contenedor
48 # Montamos el HDFS simulado en una carpeta con datos persistentes
49 VOLUME ${hdfs_simulado}
50 CMD ["bash"]
51
52 # Especificamos la ruta de trabajo dentro del contenedor
53 WORKDIR ${SPARK_HOME}
54
55 # Ejecutamos Apache Spark como nodo master
56 CMD bin/spark-class org.apache.spark.deploy.master.Master >> logs/spark-master.out
```

Spark worker

El listado 9.2 muestra el fichero Docker que permite crear las imagenes para los nodos Spark worker. Este fichero es similar al anterior pero configura la imagen para ejecutar Spark como nodo worker.

Listado 9.2: Fichero spark-worker.Dockerfile

```
1 # Selección de imagen base
2 # Especificamos como imagen base una imagen Debian ligera con Java 8 JRE
3 FROM openjdk:8-jre-slim
4
5
6 # Descarga e instalación de dependencias
7 # Definimos las variables del Dockerfile
```

```

 8 ARG hdfs_simulado=/opt/workspace
 9 ARG spark_version=3.1.2
10 ARG hadoop_version=3.2
11 ARG spark_worker_web=8081 # Puerto para interfaz web del nodo worker
12
13 # Definimos la variable de entorno conteniendo el directorio de HDFS
14 ENV HDFS_SIMULADO=${hdfs_simulado}
15
16 # Realizamos la instalación de la última versión estable de Python3
17 RUN mkdir -p ${hdfs_simulado} && \
18     apt-get update -y && \
19     apt-get install -y python3 && \
20     ln -s /usr/bin/python3 /usr/bin/python && \
21     rm -rf /var/lib/apt/lists/*
22 RUN apt-get update -y && \
23     apt-get install -y python3-pip && \
24     pip3 install gdown numpy matplotlib scipy scikit-learn
25
26 # Realizamos la instalación de Apache Spark
27 RUN apt-get update -y && \
28     apt-get install -y curl && \
29     curl https://archive.apache.org/dist/spark/spark-${spark_version}/spark-${
        spark_version}-bin-hadoop${hadoop_version}.tgz -o spark.tgz && \
30     tar -xf spark.tgz && \
31     mv spark-${spark_version}-bin-hadoop${hadoop_version} /usr/bin/ && \
32     mkdir /usr/bin/spark-${spark_version}-bin-hadoop${hadoop_version}/logs && \
33     rm spark.tgz
34
35 # Definimos las variables de entorno de Spark
36 ENV SPARK_HOME /usr/bin/spark-${spark_version}-bin-hadoop${hadoop_version}
37 ENV SPARK_MASTER_HOST spark-master
38 ENV SPARK_MASTER_PORT 7077
39 ENV PYSPARK_PYTHON python3
40
41 # Exponemos el puerto utilizado para acceder a la interfaz web del worker
42 EXPOSE ${spark_worker_web}
43
44
45 # Ejecución de comandos al arrancar el contenedor
46 # Montamos el HDFS simulado en una carpeta con datos persistentes
47 VOLUME ${hdfs_simulado}
48 CMD ["bash"]
49
50 # Especificamos la ruta de trabajo dentro del contenedor
51 WORKDIR ${SPARK_HOME}
52
53 # Ejecutamos Apache Spark como nodo worker
54 CMD bin/spark-class org.apache.spark.deploy.worker.Worker spark://${SPARK_MASTER_HOST}:${
        SPARK_MASTER_PORT} >> logs/spark-worker.out

```



Variables de entorno de Spark. El lector puede encontrar más información sobre las variables de entorno de Spark en el  Enlace: <http://spark.apache.org/docs/latest/spark-standalone.html> .

9.2. Creación de un cluster Apache Spark en Docker

JupyterLab

Finalmente, el listado 9.3 muestra el fichero Docker que permite crear la imagen para el nodo que ejecutará JupyterLab. Exponemos el puerto 8888 al exterior del contenedor, que es el que utilizaremos para conectarnos a la interfaz web de JupyterLab.

Listado 9.3: Fichero jupyterlab.Dockerfile

```
1 # Selección de imagen base
2 # Especificamos como imagen base una imagen Debian ligera con Java 8 JRE
3 FROM openjdk:8-jre-slim
4
5
6 # Descarga e instalación de dependencias
7 # Definimos las variables del Dockerfile
8 ARG hdfs_simulado=/opt/workspace
9 ARG spark_version=3.1.2
10 ARG jupyterlab_version=3.2.0
11 ARG jupyterlab_web=8888 # Puerto para interfaz web de JupyterLab
12
13 # Definimos la variable de entorno conteniendo el puerto de JupyterLab
14 ENV HDFS_SIMULADO=${hdfs_simulado}
15
16 # Definimos las variables de entorno conteniendo el directorio de HDFS
17 ENV JUPYTERLAB_PORT=${jupyterlab_web}
18
19 # Realizamos la instalación de la última versión estable de Python3
20 RUN mkdir -p ${hdfs_simulado} && \
21     apt-get update -y && \
22     apt-get install -y python3 && \
23     ln -s /usr/bin/python3 /usr/bin/python && \
24     rm -rf /var/lib/apt/lists/*
25 RUN apt-get update -y && \
26     apt-get install -y python3-pip && \
27     pip3 install gdown numpy matplotlib scipy scikit-learn
28
29 # Realizamos la instalación de la versión especificada de Pyspark y JupyterLab
30 RUN pip3 install wget pyspark==${spark_version} jupyterlab==${jupyterlab_version}
31
32
33 # Ejecución de comandos al arrancar el contenedor
34 # Montamos el HDFS simulado en una carpeta con datos persistentes
35 VOLUME ${hdfs_simulado}
36 CMD ["bash"]
37
38 # Exponemos el puerto utilizado por JupyterLab
39 EXPOSE ${jupyterlab_web}
40
41 # Especificamos la ruta de trabajo dentro del contenedor
42 WORKDIR ${HDFS_SIMULADO}
43
44 # Ejecutamos JupyterLab utilizando el puerto especificado
45 CMD jupyter lab --ip=0.0.0.0 --port=${JUPYTERLAB_PORT} --no-browser --allow-root --
    NotebookApp.token=
```



Atención. Nótese que se podrían reducir las redundancias de estas imágenes mediante la creación de imágenes intermedias con el contenido común entre las imágenes creadas (por ejemplo la instalación de Spark, Python, etc). Sin embargo, dado que el uso de Docker no es uno de los objetivos de este libro, se ha optado por el uso de menos imágenes para disminuir la complejidad.

9.2.4. Montar las imágenes de Docker

Una vez las imágenes están listas es hora de montarlas en la máquina. El listado 9.4 contiene un script en bash que monta las tres imágenes definidas anteriormente. Al montarlas, Docker descarga la imagen base que hemos especificado, y siguiendo la receta especificada en los ficheros Dockerfile que hemos definido anteriormente, instala todas las dependencias y configuraciones que hemos especificado. Notar que en lugar de utilizar el fichero `build.sh` también se podrían ejecutar cada uno de los comandos manualmente en la consola, teniendo cuidado de sustituir las variables definidas en las líneas ①-3 por el contenido correspondiente.

Listado 9.4: Fichero `build.sh`

```
1 SPARK_VERSION="3.1.2"
2 HADOOP_VERSION="3.2"
3 JUPYTERLAB_VERSION="3.2.0"
4
5 # Creación de la imagenes de Docker
6 docker build \
7   --build-arg spark_version="${SPARK_VERSION}" \
8   --build-arg hadoop_version="${HADOOP_VERSION}" \
9   -f spark-master.Dockerfile \
10  -t spark-master .
11
12 docker build \
13   --build-arg spark_version="${SPARK_VERSION}" \
14   --build-arg hadoop_version="${HADOOP_VERSION}" \
15   -f spark-worker.Dockerfile \
16   -t spark-worker .
17
18 docker build \
19   --build-arg spark_version="${SPARK_VERSION}" \
20   --build-arg jupyterlab_version="${JUPYTERLAB_VERSION}" \
21   -f jupyterlab.Dockerfile \
22   -t jupyterlab .
```

Para ejecutar el listado 9.4 y posteriormente visualizar las imágenes que tenemos instaladas en nuestra máquina, se puede ejecutar las instrucciones:

```
$ bash build.sh
$ docker images
```

9.2.5. Puesta en marcha del cluster

El último paso será crear los contenedores a partir de las imágenes previas y poner así en marcha nuestro cluster de Spark. Para ello utilizaremos un fichero de Docker compose especificado en el listado 9.5 el cual contiene la receta para poner en marcha nuestro cluster. Lo que hará será, primeramente crear un volumen compartido que simulará el sistema HDFS (líneas ③-6) y a continuación creará los contenedores especificados en la figura 9.2.

Listado 9.5: Fichero `docker-compose.yml`

```
1 version: "3.6"
2
3 volumes:
4   hdfs-simulado:
5     name: "hadoop-distributed-file-system"
6     driver: local
7
8 services:
9   spark-master:
10    image: spark-master
11    container_name: spark-master
12    ports:
13      - 7077:7077
14      - 8080:8080
15    volumes:
16      - hdfs-simulado:/opt/workspace
17
18   spark-worker-1:
19    image: spark-worker
20    container_name: spark-worker-1
21    environment:
22      - SPARK_WORKER_CORES=1
23      - SPARK_WORKER_MEMORY=512m
24    ports:
25      - 8081:8081
26    volumes:
27      - hdfs-simulado:/opt/workspace
28    depends_on:
29      - spark-master
30
31   spark-worker-2:
32    image: spark-worker
33    container_name: spark-worker-2
34    environment:
35      - SPARK_WORKER_CORES=1
36      - SPARK_WORKER_MEMORY=512m
37    ports:
38      - 8082:8081
39    volumes:
40      - hdfs-simulado:/opt/workspace
41    depends_on:
42      - spark-master
43
44   jupyterlab:
45    image: jupyterlab
```

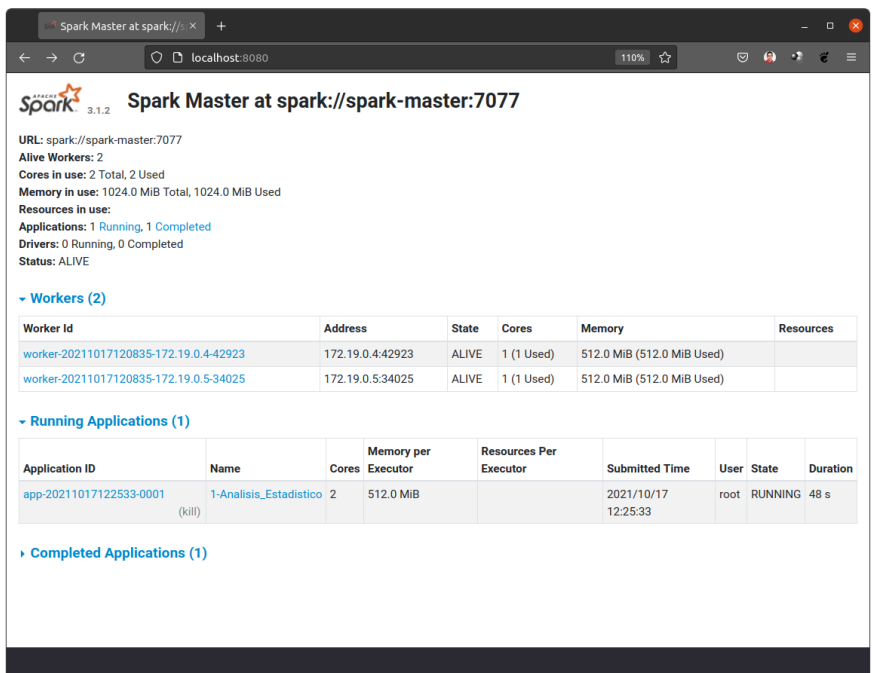


Figura 9.3: Interfaz web del nodo maestro del cluster Spark durante la ejecución de una aplicación.

```
46     container_name: jupyterlab
47     ports:
48       - 8888:8888
49     volumes:
50       - hdfs-simulado:/opt/workspace
```

Finalmente, ejecutamos la receta anterior mediante el comando:

```
$ docker-compose up
```

Para terminar la ejecución, se debe presionar las teclas `ctrl`+`c`.



Número de núcleos y memoria. Las variables `SPARK_WORKER_CORES` y `SPARK_WORKER_MEMORY` especifican el número de cores y de memoria RAM a utilizar por cada uno de los nodos worker. Se debe ajustar este valor en función de las capacidades de la máquina del usuario, de forma que no se utilicen más recursos de los disponibles.

Una vez tenemos los contenedores en ejecución, podemos conectarnos a la dirección `localhost:8080` desde un navegador web en la máquina del usuario para visualizar el panel de información del nodo Spark master (véase la figura 9.3).

9.2. Creación de un cluster Apache Spark en Docker

Para eliminar todos los contenedores de la máquina del usuario, se puede utilizar:

```
$ docker rm -vf $(docker ps -a -q)
```

Y para eliminar todas las imágenes creadas en la máquina:

```
$ docker rmi -f $(docker images -a -q)
```

Recuerda que es conveniente eliminar todos los contenedores antes de eliminar todas las imágenes a partir de las cuales se crearon esos contenedores.

9.2.6. Creación de una aplicación con PySpark

Una vez el cluster se está ejecutando, nos conectaremos a la dirección `localhost:8888` donde tenemos ejecutando la aplicación JupyterLab (vease la figura 9.4). Utilizando JupyterLab podremos crear cuadernos de Jupyter que hagan uso de la biblioteca PySpark para ejecutar en nuestro cluster. Para ello, es necesario incluir el código incluido en el listado 9.6 para conectarse al cluster que hemos creado.

Listado 9.6: Código Python necesario para crear una nueva sesión de Spark conectada al nodo Spark master

```
1 from pyspark import SparkContext, SparkConf
2
3 # Establecer las propiedades de Spark:
4 # - URL de conexión
5 # - Nombre de la aplicación
6 # - Memoria de los procesos Spark worker
7 conf = SparkConf().setMaster("spark://spark-master:7077").setAppName("Mi-primer-a-
    aplicacion-en-Spark").set("spark.executor.memory", "512m")
8
9 # Iniciar un cluster de Spark (comprueba si ya existe un cluster con esta configuración
    y en el caso contrario crear uno nuevo)
10 sc = SparkContext.getOrCreate(conf=conf)
11
12 # Mostramos el cluster creado
13 display(sc)
14
15 # A continuación irá el código de nuestra aplicación ...
```

Una vez hemos creado nuestro propio cluster de Spark, ya estamos listos para pasar al análisis de datos. Todas las técnicas descritas en las siguientes secciones podrán ser programadas en Spark usando PySpark. La API de Pyspark puede consultarse en el  Enlace: <http://spark.apache.org/docs/latest/api/python> .

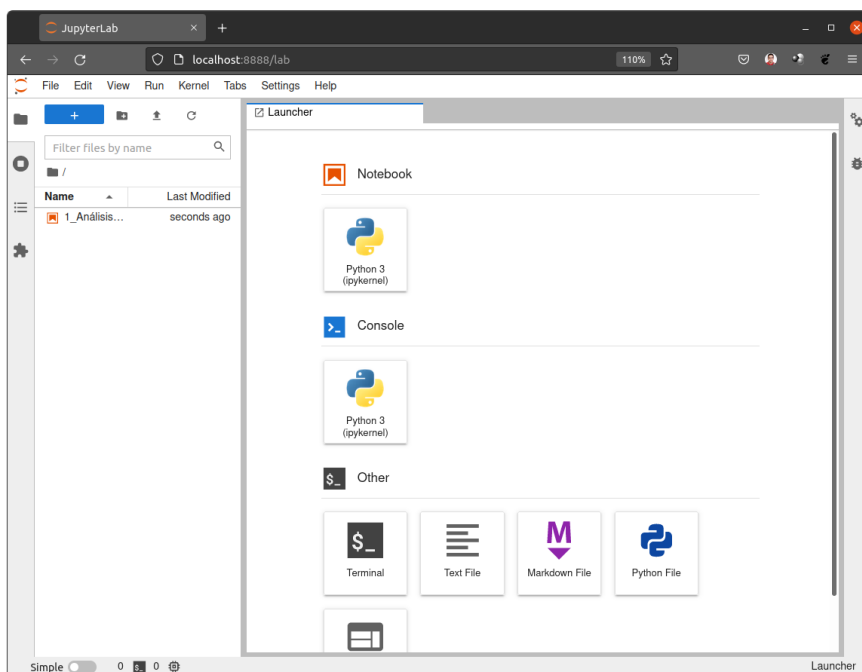


Figura 9.4: Interfaz web del servidor de JupyterLab instalado en nuestro cluster Spark.

9.3. Análisis de datos en Big Data

Cuando hablamos del análisis de datos en Big Data, hablamos de la combinación de los enfoques tradicionales de análisis de datos estadísticos con enfoques computacionales. Si se dispone de la totalidad del dataset, pueden aplicarse técnicas tradicionales de muestreo estadístico de la población. Es decir, tomar un subgrupo de los datos que sea representativo de toda la población. La muestra debe tener un tamaño suficiente para garantizar un análisis estadístico adecuado de estos datos. Este enfoque es típico de los escenarios de procesamiento por lotes (o *batch*). Sin embargo, en Big Data podemos cambiar este procesamiento por lotes por el procesamiento en tiempo real cuando las necesidades del problema así lo requieran. Por ejemplo, en un contexto donde los datos nos llegan en flujo constante, el conjunto de datos se va acumulando con el tiempo, y los datos siguen un orden temporal. En este contexto, es interesante poner el énfasis en el procesamiento en tiempo real (o *on-line*), ya que los resultados analíticos pueden tener una vida útil y es necesaria la toma de decisiones rápidamente, por lo que no se puede esperar a su completa recolección para realizar un procesamiento por lotes.

Un sistema que requiera procesamiento de datos en tiempo real puede dividirse en sistemas de tiempo real fuerte, en los cuales no satisfacer una fecha límite provocaría un fallo completo del sistema, y en sistemas de tiempo real suave, dónde no cumplir el plazo límite indica que el sistema no está funcionando de manera óptima. Sin embargo, en sistemas que requieran un procesamiento por lotes, no cumplir con el plazo límite de procesamiento podría indicar que el sistema necesita más capacidad de cómputo para poder realizar las tareas en el tiempo deseado.

A continuación se presenta un ejemplo de cada tipo de procesamiento. Un ejemplo de sistema que requiere un procesamiento por lotes es el de una empresa que se encargue de realizar la venta de equipamiento electrónico a nivel mundial. En este caso, todos los datos relativos a las ventas realizadas en un día se almacenan en un *Data Lake* donde se preservan hasta ser requeridos para un procesamiento posterior. Por ejemplo, cada madrugada el sistema puede realizar un análisis de las ventas del día anterior en cada una de las tiendas, gestionando de esta manera el inventario, y permitiendo que los directivos tengan los reportes de ventas a nivel mundial a primera hora de la mañana.

Un ejemplo de un sistema que requeriría un procesamiento de datos en tiempo real es el sistema de control de una fábrica. En la cadena de producción se localizarían diversos dispositivos de control, así como las diferentes maquinarias, que enviarían un flujo constante al servidor central que se encarga de procesarlos. En este contexto, se requiere un procesamiento prácticamente inmediato de los datos, que permita que el sistema tome decisiones en tiempo real y permita la intervención en el proceso de producción para asegurar la calidad del producto, así como la protección del equipamiento. En este contexto, un retraso en el procesamiento puede ser catastrófico para la seguridad de las máquinas e incluso de los trabajadores. Póngase el caso del sistema de control de una central nuclear. En este tipo de arquitecturas, nos encontramos además con sistemas redundados para prevenir fallos.

Para realizar un análisis exploratorio de los datos, se suelen preferir las técnicas estadísticas. Tras ello, se pueden aplicar técnicas computacionales que aprovechen la información obtenida del estudio estadístico del conjunto de datos. El paso del análisis por lotes al tiempo real presenta otros retos, ya que las técnicas en tiempo real deben aprovechar algoritmos de elevada eficiencia computacional que permitan obtener los resultados en el momento oportuno. Por ejemplo, ¿qué pasaría si el análisis de datos en el sistema de control de una central nuclear no se realiza de manera eficiente, obteniendo los resultados con retraso y evitando que se puedan tomar acciones correctivas en el momento adecuado? Esto llevaría a un fallo completo del sistema, con todos los problemas derivados de este.

A continuación se presentan las principales técnicas de análisis de datos empleadas de manera tradicional. Nos centraremos fundamentalmente en las técnicas estadísticas, las cuales son fundamentales para garantizar que se extraiga información significativa y precisa de los datos en el entorno Big Data. Estas técnicas incluyen desde la obtención de estadísticos descriptivos básicos, métodos de muestreo de datos (especialmente útil cuando tenemos un gran volumen de estos), medición

de la incertidumbre, inferencia estadística, obtención de la correlación entre las variables de los datos, etc. Estas técnicas nos permitirán obtener una primera visión de los datos y determinar posteriormente que técnicas de visualización serán las más efectivas.

9.3.1. Análisis cualitativo

El análisis cualitativo es una técnica de análisis de datos que busca principalmente responder a preguntas como “por qué”, “qué” o “cómo”. Para ello, se centra en la descripción de diversas cualidades de los datos mediante palabras, ya sea en forma de textos o narraciones, aunque también pueden incluir recursos de audio o vídeo. Para realizar este análisis se debe examinar una muestra pequeña con profundidad. Los resultados de estos análisis no se pueden generalizar a todo el conjunto de datos debido al pequeño tamaño de esta muestra. Además, tampoco pueden medirse numéricamente, y por lo tanto no pueden utilizarse para realizar comparaciones numéricas.

Los datos usados para realizar este análisis son a menudo muy heterogéneos. La información suele ser obtenida a través de distintas técnicas, siendo las principales:

- Observaciones directas.
- Consulta de documentos públicos o privados.
- Encuestas.
- Entrevistas.
- Comunidades online o redes sociales.

En los últimos años han adquirido gran importancia el uso de datos provenientes de comunidades online o redes sociales. Pongamos un ejemplo de análisis cualitativo basado en datos obtenidos usando esta última técnica. Imaginemos una marca de ropa de moda que tiene un gran número de seguidores en las redes sociales. Esta empresa en un momento determinado, para enfocar una determinada campaña publicitaria, podría hacerse preguntas como: ¿Qué causas sociales despiertan más entusiasmo entre los adolescentes? o ¿Qué tipo de música es la más escuchada entre sus seguidores?

Para lograr esto, las empresas aprovechan las capacidades del Big Data y suelen recurrir a un gran número de bots que rastrean todas las redes sociales utilizadas por la empresa en busca de publicaciones de sus seguidores. A continuación, se realiza un análisis en profundidad sobre el contenido que consumen los seguidores, y como estos reaccionan con la marca. Toda esta información se puede utilizar para una gran cantidad de propósitos. Por ejemplo, para orientar acciones comerciales, en las cuales se quiera poner de manifiesto la responsabilidad social de la empresa hacia las causas que estén más en consonancia con las inquietudes de sus clientes.

9.3.2. Análisis cuantitativo

El análisis cuantitativo es una técnica de análisis de datos que busca cuantificar los patrones y las correlaciones encontradas en los datos. Generalmente, y a diferencia de la técnica anterior, este análisis se mide mediante el uso de números. Esta técnica utiliza técnicas estadísticas ya que implica el análisis de un gran número de observaciones provenientes de del dataset. Como el tamaño de la muestra es grande, los resultados obtenidos de aplicar esta técnica pueden aplicarse de manera generalizada a todo el conjunto de datos.

Retomando el ejemplo anterior de la marca de ropa de moda, supongamos que esta marca almacena todos los pedidos realizados a lo largo de un día (ya sea desde su página web, o los distintos distribuidores) en un data lake para su posterior procesamiento. Realizando un Análisis cuantitativo de estos datos, la marca puede descubrir que en Junio se produce un aumento de un 35 % de las ventas de determinados colores veraniegos, o por ejemplo que una disminución del 20 % el el precio de cierta prenda, aumenta sus ventas en un 40 %. El resultado de este análisis se puede extrapolar al total de las ventas y puede ser de gran utilidad para tomar decisiones de negocio.

9.3.3. Análisis estadístico

El análisis estadístico utiliza métodos estadísticos que están basados en fórmulas matemáticas como medio para analizar los datos. El análisis estadístico suele ser cuantitativo, pero también puede ser cualitativo. Este tipo de análisis se utiliza habitualmente para describir conjuntos grandes de datos y proporcionar una visión general sobre estos, por ejemplo, proporcionando la media, la mediana o la moda de los atributos asociados al conjunto de datos. También puede utilizarse para inferir patrones y relaciones dentro del conjunto de datos, como la correlación y la regresión.

A lo largo de este apartado realizaremos un breve repaso a algunos términos comúnmente utilizados en el análisis estadístico. Estos términos nos permitirán realizar un análisis general de los datos, por lo tanto, se les suelen denominar estadísticos descriptivos.

- **Población.** Conjunto de todos los elementos o observaciones que se quieren estudiar. Estos elementos pueden ser objetos, acontecimientos, sucesos, grupos de personas, registros de un sistema, etc. Uno de los objetivos más comunes de las técnicas estadísticas es obtener información relevante sobre la población elegida.
- **Muestra.** Subconjunto de elementos de una población. En determinadas situaciones, es muy complicado realizar un estudio estadístico sobre todos los elementos de la población, por lo tanto se toma una muestra que sea representativa de esta población y se realiza el estudio sobre ella. Es importante que la técnica de muestreo sea la adecuada para que se produzca una muestra

representativa de la población, con las mismas características que esta. Si la técnica no es la adecuada, se puede obtener una muestra sesgada de la población, lo cual limita el interés y el uso de la información obtenida. Cualquier valor calculado a partir de una muestra se denomina un estadístico muestral.

Dada una población o una muestra, es interesante calcular el promedio de estos valores, al que también se le conoce como medida de tendencia central. Un promedio es un valor típico o representativo de un conjunto de datos. Pueden definirse varios tipos de promedios, los más utilizados son la media aritmética, media geométrica, media armónica, mediana y la moda.

- **Media aritmética.** Es el valor característico de una serie de datos cuantitativos que se obtiene a partir de la suma de todos los valores una serie de datos dividida entre el número de sumandos (la cantidad de datos en la serie). Dada una serie de n números $\{x_1, x_2, \dots, x_n\}$ la media aritmética de todos ellos se define mediante la expresión:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i = \frac{x_1 + x_2 + \dots + x_n}{n}.$$

- **Media geométrica.** Es el valor característico de una serie de datos cuantitativos que se obtiene al calcular la raíz n -ésima del producto de todos los números de una serie de datos. Uno de los principales usos de la media geométrica es el cálculo de medias sobre porcentajes, ya que su cálculo da resultados más ajustados a la realidad. Notese que todos los valores de la serie se multiplican entre sí, de modo que si alguno de ellos fuera cero, el producto total sería cero. Dada una serie de n números $\{x_1, x_2, \dots, x_n\}$ la media geométrica de todos ellos se define mediante la expresión:

$$\bar{x} = \sqrt[n]{\prod_{i=1}^n x_i} = \sqrt[n]{x_1 \cdot x_2 \cdot \dots \cdot x_n}.$$

- **Media armónica.** Es el valor característico de una serie de datos cuantitativos que se obtiene al calcular el recíproco (o inverso) de la media aritmética de los recíprocos de todos los valores una serie de datos. Este tipo de media suele utilizarse cuando los datos que se quieren promediar miden velocidades o tiempos, aunque su uso no está muy extendido. Dada una serie de n números $\{x_1, x_2, \dots, x_n\}$ la media armónica de todos ellos, denotada H , se define mediante la expresión:

$$H = \frac{n}{\sum_{i=1}^n \frac{1}{x_i}} = \frac{n}{\frac{1}{x_1} + \frac{1}{x_2} + \dots + \frac{1}{x_n}}.$$

Notese que la media armónica no está definida si alguno de los valores es cero.

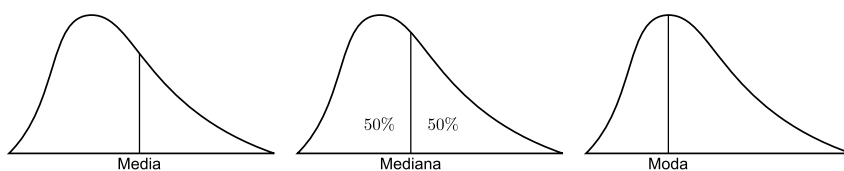


Figura 9.5: Media, mediana y moda.

- **Mediana.** Es el valor que ocupa la posición central de todos los valores en una serie de datos cuando estos están ordenados de menor a mayor. En el supuesto de que la serie tenga un número impar de elementos la mediana es el elemento central. Sin embargo, en el supuesto de que la serie tenga un número par de elementos la mediana es la media aritmética entre los dos elementos en las posiciones centrales. Notar que, además de en variables cuantitativas, la mediana también se puede calcular en las variables cualitativas ordinales (es decir, que se pueden ordenar).
- **Moda** Es el valor que aparece con más frecuencia en una serie de datos. En el supuesto de que una serie de datos tenga dos elementos que aparezcan repetidos con la misma frecuencia y esa frecuencia sea máxima (moda), entonces decimos que la serie tiene una distribución bimodal (o multimodal en el caso de que existan más de dos modas en la serie). Sin embargo, si todos los elementos de la serie tienen la misma frecuencia, entonces decimos que la serie no tiene moda. Notar que el concepto de moda puede aplicarse tanto para variables cualitativas como cuantitativas.

La Figura 9.5 muestra de manera gráfica los conceptos de media, mediana y moda en una serie de datos.



Cuando los datos tienen naturaleza discreta, por ejemplo la variable x codifica numéricamente el nombre de un país (1 Andorra, 2 Angola, etc.) las anteriores medidas no tienen significado (¿Qué país es 24,67?). Para variables cualitativas una medida de centralidad es la **moda**, que es el valor más repetido.

- **Desviación estándar.** Es una medida estadística de dispersión, es decir, se utiliza para cuantificar qué tan dispersos están los datos con respecto a la media. A mayor valor de la desviación estándar, mayor será la dispersión de los datos, es decir, más se alejarán de la media. Sin embargo, una desviación estándar baja indica que la mayor parte de los datos se agrupan cerca de la media. La desviación estándar se denota tradicionalmente con la letra griega σ , y en determinados libros también se la denomina con el nombre de desviación típica.

Existen varias formas de calcular la desviación estándar de un conjunto de datos. Una de las expresiones más utilizadas es la raíz cuadrada de la varianza de este conjunto de datos. La **varianza**, representada como σ^2 , de un conjunto de datos es otra medida de dispersión que representa la variabilidad de una serie de datos con respecto a su media. La varianza se define mediante la expresión:

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2.$$

Por lo tanto, a partir de la definición anterior, la expresión de la desviación estándar será:

$$\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2}.$$

Notar a partir de la expresión anterior que el valor de la desviación estándar será siempre mayor o igual que cero.

En determinadas ocasiones realizar un estudio sobre una población puede ser algo muy complicado de conseguir. Imaginemos que queremos conocer el número medio de coches que tiene cada familia en España. En principio es posible conocer este valor, solo tendríamos que hacer una encuesta entre todas las familias en España. Sin embargo, esto no es algo factible de llevar a la realidad. En estos casos, es mucho más accesible realizar una encuesta sobre una muestra de dicha población. Por ejemplo, imagina una muestra de 10,000 personas elegidas de manera aleatoria en todo el territorio. Sobre esta muestra es sencillo calcular medidas como la media de la muestra o la desviación estándar. No obstante, seguimos sin saber el verdadero valor de la población.

Veamos como podemos aplicar todos los conceptos anteriores para realizar una inferencia estadística. Una inferencia estadística es un conjunto de métodos que permiten inducir a partir de una muestra estadística cuál es el comportamiento de una población bajo estudio. En este caso, vamos a estudiar una técnica de inferencia estadística que se denomina **intervalo de confianza**. Esta técnica estadística nos permite, a partir de una muestra, acotar dos valores (un intervalo) entre los cuales se encontrará con una determinada probabilidad la media de una población.

Si el tamaño de la muestra es lo suficientemente grande o la población sigue una distribución normal, y conociendo desviación estándar, la distribución de la media muestral es, prácticamente, una distribución normal de media μ y desviación estándar $\frac{\sigma}{\sqrt{n}}$, donde n es el tamaño de la muestra. Podemos estandarizar esta distribución, es decir, convertirla en una distribución normal de media cero y desviación estándar uno de la siguiente manera:

$$\frac{\bar{x} - \mu}{\sigma/\sqrt{n}} \sim \mathcal{N}(0, 1),$$

donde $\bar{x}$ es la media de la muestra, μ es la media de la población (que se quiere determinar), σ es la desviación estándar y $\sqrt{n}$ es la raíz cuadrada del tamaño de la muestra. Se quiere calcular un intervalo de confianza para la media muestral μ con un nivel de confianza $1 - \alpha$. Normalmente se utilizan valores de confianza del 95 % y del 99 %. Por lo tanto α es el error que se cometerá, es decir, el término opuesto.

La probabilidad de que la media poblacional se encuentre en un intervalo concreto con un nivel confianza $1 - \alpha$ es:

$$P(\bar{x} - Z_{\alpha/2} \cdot \sigma / \sqrt{n} < \mu < \bar{x} + Z_{\alpha/2} \cdot \sigma / \sqrt{n}) = 1 - \alpha,$$

donde $Z_{\alpha/2}$ es el valor en una distribución normal, que dejaría a la derecha un área bajo la curva de $\alpha/2$. Por ejemplo, para una confianza del 95 %, debemos considerar que queda un área de $\frac{0,05}{2} = 0,025$ a cada extremo de la curva. El valor que deja a la derecha un área bajo la curva de 0,025 es 1,96 (véase la Figura 9.6).

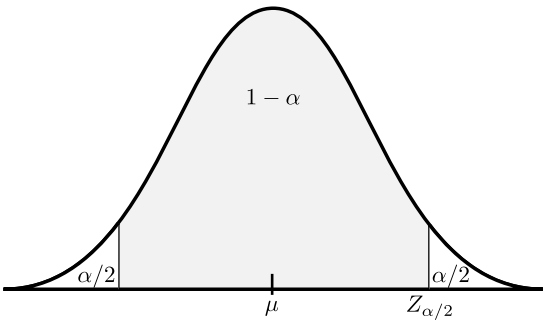


Figura 9.6: Área bajo la curva normal.



Ejercicio. Dada la siguiente muestra sobre las alturas de los alumnos en un curso de Big Data {1,65; 1,70; 1,85; 1,92; 1,58; 1,69; 1,72; 1,62; 1,90; 1,74} calcula el intervalo de confianza al 95 % para la media. Repite el ejercicio anterior considerando ahora un intervalo de confianza al 99 %.



Inferencia estadística. Cuando trabajamos en Big Data, puede ser muy costoso realizar algún tipo de análisis sobre todos los datos disponibles para obtener un indicador de interés. En este caso puede ser una buena idea calcular este indicador sobre una muestra de la población, y a continuación aplicar una técnica de inferencia estadística como el intervalo de confianza para acotar los valores más probables entre los que se encontrará el indicador deseado para toda la población.

Una vez presentados los principales estadísticos descriptivos, a continuación vamos a estudiar como podemos utilizar análisis estadísticos para inferir patrones y relaciones entre los datos.

9.3.4. Correlación

Otro concepto estadístico importante en el análisis de datos es el concepto de correlación. La correlación es una técnica que nos permite determinar si dos variables están relacionadas entre sí y en que grado. Si comprobamos que dos variables están relacionadas (están correlacionadas), el siguiente paso será determinar cuál es su relación.

El uso de la correlación en el análisis de datos nos permitirá comprender el dataset y extraer relaciones que nos ayudarán a explicar los fenómenos que puedan darse entre las variables. Esta es por tanto una técnica muy importante en la minería de datos, usada para identificar relaciones entre las variables del dataset, identificación de patrones y anomalías.

La correlación entre dos variables se expresa como un número decimal en el rango $[-1, 1]$ al cual se le conoce como **coeficiente de correlación**. Cuando la correlación es positiva, indica que las dos variables aumentan al mismo tiempo. Y si la correlación es negativa significa que cuando una de las variables aumenta la otra disminuye y viceversa. Cuanto más se acerque el coeficiente de correlación a 1, mayor será la correlación positiva y cuanto más se acerque a -1 , mayor será la correlación negativa. Un valor de correlación igual o cercano a 0 significa que no hay correlación entre las variables.

Existen varios tipos de coeficientes de correlación, pero destaca el **coeficiente de correlación de Pearson**. Este coeficiente funciona bien con variables cuantitativas con una distribución normal. No obstante, según [McD09] este sigue siendo robusto pese a la falta de normalidad, aunque es sensible a valores extremos (outliers). El coeficiente de Pearson para una población puede calcularse mediante la expresión

$$r_{xy} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}},$$

dónde n es el tamaño de la muestra, x_i, y_i son los puntos muestrales de la primera y segunda variable respectivamente y $\bar{x}, \bar{y}$ son las medias muestrales de la primera y segunda variable respectivamente.

La Figura 9.7 muestra un ejemplo de dos variables con correlación positiva, sin correlación y correlación negativa (de izquierda a derecha). Los coeficientes de Pearson de cada una de las gráficas son 0,89, $-0,05$ y $-0,89$, respectivamente.

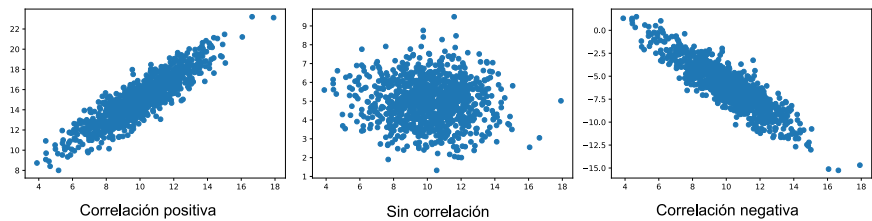


Figura 9.7: Ejemplo de dos variables con correlación positiva, sin correlación y correlación negativa.



Pregunta. Imaginemos que estamos analizando los datos de venta de una heladería. Nos podríamos preguntar si la temperatura en la ciudad donde está situada la heladería influye en el número de ventas de helados o si el tamaño de estos helados influye en sus ventas. Este es un claro ejemplo de preguntas que pueden ser respondidas usando correlación.

9.3.5. Regresión

La regresión es una técnica que nos permite explorar como una variable dependiente se relaciona con otra variable independiente dentro de un conjunto de datos determinado. En el ejemplo de la heladería, la regresión nos permitiría determinar que tipo de relación existe entre la temperatura y el número de ventas de helados.

La regresión y la correlación son dos técnicas estrechamente relacionadas. El análisis de la correlación produce un número que resume el grado de correlación entre dos variables, mientras que el análisis de la regresión produce una ecuación matemática que describe la relación entre ambas variables.

Uno de los tipos más sencillos de regresión es la regresión lineal simple. La regresión lineal consiste en generar un modelo de regresión (la ecuación de una recta) que permita describir la relación lineal existente entre dos variables. Si denominamos Y a la variable dependiente y denominamos X a la variable independiente, entonces tenemos la siguiente ecuación:

$$Y = \beta_0 + \beta_1 X + \epsilon,$$

dónde β_0 es la ordenada en el origen, β_1 es la pendiente de la recta y ϵ es el término de error. Los dos primeros términos deben ser estimados y se les conocen como coeficientes de regresión. El término de error (también denominado residuo) representa la diferencia entre el valor ajustado por la recta y el valor real de la variable dependiente.

En este punto, el lector se preguntará cómo podemos obtener las estimaciones de β_0 y β_1 . Existen diferentes alternativas, pero en este apartado vamos a explicar uno de los métodos más conocidos, denominado método de mínimos cuadrados, que consiste calcular aquellos valores de los parámetros que minimizan la suma de

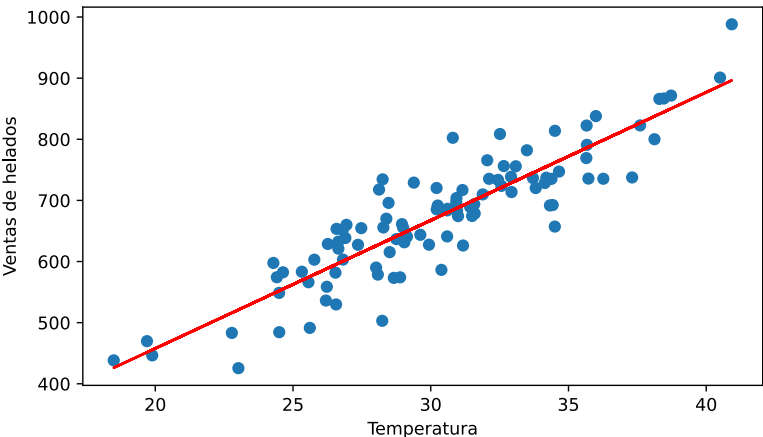


Figura 9.8: Número de ventas de helados con respecto a la temperatura de la ciudad.

los cuadrados de los residuos. Con este método, las expresiones para calcular β_0 y β_1 son las siguientes:

$$\beta_1 = \frac{S_Y}{S_X} R,$$
$$\beta_0 = \bar{y} - \beta_1 \bar{x},$$

dónde $\bar{x}$ y $\bar{y}$ denotan las medias muestrales de X e Y , respectivamente, S_X y S_Y son las desviaciones típicas de cada variable, y finalmente R es el coeficiente de correlación entre ambas.

La Figura 9.8 muestra un ejemplo de regresión lineal sobre una serie de datos que muestra el número de helados vendidos por una heladería y la temperatura de la ciudad en el momento de la venta. La línea roja representa la recta ajustada utilizando regresión lineal simple.

La regresión y la correlación tienen una serie de diferencias importantes. La correlación no implica causalidad, es decir, el cambio en el valor de una variable podría no ser la causa del cambio en el valor de la segunda variable, aunque ambas cambien al mismo ritmo. Esto puede ocurrir debido a una tercera variable desconocida, conocida como factor de confusión. La correlación supone que ambas variables son independientes. La regresión, en cambio, se aplica a dos variables que han sido previamente identificadas una como independiente y la otra como dependiente. Por ello, implica que existe un grado de causalidad entre las variables.

Dentro de Big Data, la correlación puede aplicarse primero para descubrir si existe una relación. A continuación, se puede aplicar la regresión para seguir explorando la relación y predecir los valores de la variable dependiente, basándose en los valores conocidos de la variable independiente.